

Forensic Trajectory Analysis and Diagnostic Modeling for Local Coding Agents: Architectures, Empirical Failure Modes, and Observability Paradigms

Technical Specifications of Local Coding Agent Traces

Directory Topography and Storage Architectures

Local AI coding agents maintain complete, step-by-step execution records as structured, append-only JSON Lines (JSONL) transcripts on disk.¹ These files are stored within a project-centric directory hierarchy that links local software workspaces directly to unique session buckets.¹ Taking the Claude Code agent platform as a primary example, session directories are constructed deterministically relative to the user's home directory.²

The primary configuration paths map as follows:

- **Session Transcripts:** Files are written to `~/.claude/projects/<encoded-cwd>/sessions/<session-uuid>.jsonl`.² The project path component is derived as a URL-encoded string of the absolute repository path, such as `-home-user-myapp`.²
- **Task Logs:** Incremental sub-steps are registered under `~/.claude/tasks/<session-id>/<step-number>.json`.²
- **Durable Architectural Plans:** Declarative plans reside at `~/.claude/plans/<plan-name>.md`.²
- **Team Orchestrations:** Collaborative multi-agent policies are parsed from `~/.claude/teams/<team-name>.json`.²

This file-system topology enables offline execution indexing without database dependencies.²

The files grow dynamically as the agent interacts with the workspace.² The file-system watcher mechanisms of downstream analytical engines can watch these directories to reconstruct conversations and execution state on demand.¹

JSON Record Schema and Envelope Structure

Each line in a session transcript is a single JSON record representing an atomic event, such as a user prompt, private thinking block, or tool execution.² Records are wrapped in a standardized envelope that includes routing, temporal, and spatial metadata²:

JSON

```
{  
  "type": "assistant",  
  "uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",  
  "parentUuid": "1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d",  
  "timestamp": "2025-02-20T09:14:32.441Z",  
  "sessionId": "abc123",  
  "cwd": "/home/user/myapp",  
  "message": { "role": "assistant", "content": "...",  
  }
```

The parentUuid property is a critical architectural element.² Rather than treating transcripts as linear arrays, this parent-child linkage constructs a Directed Acyclic Graph (DAG).² When an agent branches to evaluate alternative implementations, retries failed terminal queries, or spawns specialized subagents, each branch retains its distinct line of descent.² This DAG structure allows analytical engines to trace execution histories and state changes accurately.² The message payloads within this envelope are classified into seven core execution types:

- **User Records:** Represent direct user inputs, shell execution returns, file-system status updates, or injected context hooks.²
- **Assistant Records:** Represent the agent's response, typically containing private thinking blocks alongside tool invocation arrays.²
- **Tool Call and Tool Result Records:** Track the input parameters, execution latencies, and returns of shell operations, file searches, and editing tasks.²

Standardization of AI Attribution and Provenance

To decouple trace telemetry from vendor-specific formats, open standards such as the Agent Trace specification are establishing standard schemas for attributing AI contributions in repositories.⁵ Under the Agent Trace v0.1.0 RFC, trace records are registered under the MIME type application/vnd.agent-trace.record+json.⁶ These records bind directly to version control system (VCS) revisions, such as Git commit SHAs, Jujutsu stable change IDs, or Mercurial changeset hashes.⁶

This specification maps code provenance down to precise line numbers.⁶ The trace record schema maintains an array of file objects relative to the repository root.⁶ Each file object lists the conversational trajectories that modified its contents.⁶ Conversational records contain look-up URIs referencing the original execution logs, the default contributor profile (human, AI, or mixed), and an array of range definitions.⁶ Line ranges are declared using 1-indexed start and

end bounds, augmented by an optional content hash.⁶ This hash allows systems to track code attribution even when subsequent refactoring shifts line boundaries.⁶ Integrating these open tracing protocols into continuous integration pipelines allows organizations to inject persistent metadata into commits via Git trailers, maintaining an audit trail from the production codebase back to the exact developer session that generated the logic.⁷

Technical Parameter	Claude Code Session Format	Agent Trace v0.1.0 RFC Specification
Primary File Location	~/.claude/projects/<encoded-cwd>/sessions/<uuid>.jsonl	Standardized repository-relative metadata file
Logical Representation	Directed Acyclic Graph (DAG) using parentUuid references	File-to-conversation mapping object
Line-Level Attribution	Not supported natively; requires post-processing	Explicit ranges array with start_line and end_line
VCS Synchronization	Captured as loose metadata in raw transcript lines	Explicit binding to Git commit SHAs or Jujutsu stable change IDs
MIME Type Definition	Standard application/x-ndjson or flat .jsonl	Standardized as application/vnd.agent-trace.record+json
Primary Use Case	In-flight execution logs and state restore checkpoints	Downstream compliance, auditing, and provenance tracking

Empirical Foundations of Agent Failure Modes and Trajectory Datasets

Behavioral Patterns in SWE-chat

Evaluating how coding agents operate in production requires moving beyond curated benchmarks to analyze real-world usage datasets.⁸ The SWE-chat dataset, compiled by Stanford University, provides a large-scale empirical baseline consisting of 6,000 real

developer sessions captured via Entire.io integration.⁸ Comprising over 63,000 user prompts and 355,000 agent tool calls across more than 200 repositories, the dataset documents how developers interact with agents and how agents fail in natural workflows.⁸

The dataset reveals that agent-assisted coding patterns are highly bimodal.⁸ In approximately 41% of recorded sessions, developers engage in "vibe coding," wherein the AI agent authors more than 99% of the committed codebase.⁸ Conversely, in 23% of sessions, humans write all code themselves, treating the agent merely as a passive search and retrieval assistant.⁸ This bimodal split highlights a massive efficiency gap⁸:

- Only 44% of all agent-generated code survives into final user commits.⁸
- Agent-written code introduces a higher density of security vulnerabilities than human-authored equivalents.⁸
- Users actively push back against agent outputs in 44% of all turns through real-time corrections, failure reports, and abrupt session interruptions.⁸

Operational Metric	Observed Value in SWE-chat
Total Recorded Sessions	6,000
User Prompt Count	63,000+
Agent Tool Call Count	355,000
Vibe Coding Session Prevalence	41%
Human-Led Coding Session Prevalence	23%
AI Code Survival Rate in Commits	44%
User Pushback Frequency per Turn	44%
User Interruptions per Turn	5%
User Corrections/Failures per Turn	39%

Categorization of Feedback-Based and Within-Turn Failures

Analyzing low-success sessions (rated between 2 and 15 out of 100) indicates two dominant failure categories: feedback-based failures and within-turn failures.⁹ Feedback-based failures are identified by parsing the human operator's response to an agent turn, which serves as a

natural signal of dissatisfaction.

The primary feedback indicators are:

- **User Corrections:** Direct statements pointing out syntax errors, misaligned paths, or logical bugs.⁹
- **Failure Reports:** System crash logs or test suite failures copy-pasted back to the agent.⁹
- **Hard User Interruptions:** Terminating an agent mid-turn when its trajectory suggests unproductive exploration.⁹

Conversely, within-turn failures represent problematic or inefficient behavior within a single agent turn, independent of subsequent human feedback.

These are characterized by:

- **Tool Call Loops:** Repeating identical command runs or file searches without resolving the underlying block.¹⁰
- **Argument Misalignments:** Utilizing outdated or syntactically incorrect parameters in tool invocations.¹⁰
- **Unproductive Explorations:** Executing extensive directory scans or file edits completely unrelated to the user's actual prompt, often driven by incorrect context assembly.⁹

Preprocessing, Compaction, and Retrieval Architectures

Conversational Compilers and Token Reduction

As agent executions extend across long-horizon sessions, maintaining the complete conversation history degrades model response quality and increases API costs.¹¹ To address this, the View-oriented Conversation Compiler (VCC) introduces a formal compiler architecture (lexing, parsing, intermediate representation, lowering, and emitting) to compress raw agent JSONL logs into structured views.¹¹ Rather than feeding a flat text stream or raw JSON to an evaluator, VCC compiles logs into three distinct representations¹¹:

1. **Full View:** A lossless transcript serving as the canonical line-number coordinate system.¹¹
2. **User-Interface View:** A view reconstructing the interaction as the human operator perceived it.¹¹
3. **Compacted View:** An active pointer view mapping high-level actions to raw source blocks.¹¹

VCC optimizes token consumption by executing deterministic, trace-specific transformations.¹² For example, tool parameters are compiled from escaped JSON into clean, readable YAML block scalars, and system-injected XML wrappers are programmatically stripped.¹²

By compiling logs into these structured formats, the token consumption of evaluator models is reduced mathematically:

$$C_{\text{ratio}} = 1 - \frac{\sum_{i=1}^n T_{\text{view}}(i)}{\sum_{i=1}^n T_{\text{raw}}(i)}$$

On typical software engineering tasks, VCC achieves a context compression ratio

$C_{\text{ratio}} \in [0.50, 0.67]$, cutting token consumption by half to two-thirds.¹¹ This structural compression directly benefits downstream performance; evaluations on the AppWorld benchmark demonstrate higher task completion rates and more concise memory generation when models analyze compiled VCC views rather than raw JSONL dumps.¹¹

For local, resource-constrained environments, pi-vcc implements an algorithmic conversation compactor designed specifically for the Pi agent platform.¹³ Unlike LLM-based summarization calls, which are non-deterministic, slow, and expensive, pi-vcc runs fully algorithmically on local CPUs with a latency of 30 to 470 milliseconds, achieving token reductions of 35% to 99%.¹³

To preserve historical context across compactions, pi-vcc employs a structured merge policy¹³.

- **Concise Sticky Sections:** Maintains the overarching session goal and user preference variables.¹³
- **Fresh-Only Replacement:** Replaces the outstanding context node on every turn.¹³
- **Set Union Operations:** Combines file changes, edits, and Git commits uniquely across execution boundaries.¹³
- **Rolling Lossy Window:** Drops older lines from the brief transcript view while keeping active history searchable via a local regex-enabled utility (vcc_recall) that parses the raw local JSONL source on demand.¹³

Suspicious Region Retrieval Pipelines

To run diagnostic evaluations over long-horizon traces efficiently, observability platforms use a two-tier retrieval architecture to find "suspicious regions" before loading full context arrays¹⁰:

1. **Skeleton Ingestion:** This initial phase parses dense execution traces into skeleton trajectories.¹⁰ These skeletons store only basic attributes per turn: message role, execution latency, and character payload sizes, completely omitting the heavy content blocks.¹⁰
2. **Screening Routing:** A lightweight screener model (such as Claude Haiku) scans these skeletons in parallel blocks of 20 to detect anomalies, such as repetitive tool calls or spike latencies.¹⁰ The screener returns a structured list containing the flagged trace ID, error category, and a brief description.¹⁰ Full-text context retrieval is then targeted strictly at these flagged regions, reducing the search space and saving token costs.¹⁰

Diagnostic Modeling, Failure Onset Localization, and Process-Level Debugging

Hierarchical Trace Trees and Failure Localization

Identifying when and why a coding agent first went off-course is exceptionally difficult due to error compounding, where early mistakes cascade into fundamental failures.¹⁵ The CodeTracer framework addresses this by performing "failure onset localization" over complex, multi-stage workflows.¹⁵

CodeTracer reconstructs execution histories into a hierarchical trace tree.¹⁵ Each step in the trajectory is parsed into normalized records containing actions, diffs, and verification outcomes.¹⁵ These records are then classified into two node categories¹⁵:

- **Exploration Nodes:** Actions that only inspect the repository (such as searching text, listing directories, or running status checks) are clustered under the current active state.¹⁵ These nodes preserve the context and do not trigger transition branches.¹⁵
- **State-Changing Nodes:** Actions that modify the codebase or runtime environment (such as editing files, installing packages, or executing build scripts) trigger state-changing transitions to child nodes.¹⁵

By traversing this hierarchical state tree, CodeTracer's diagnostic engine can locate the earliest causal error, producing a localized diagnostic payload that can be fed back into the agent via reflective replay, allowing it to recover from past mistakes and complete previously failed runs.¹⁵

Interactive Trajectory Debugging

While post-hoc frameworks handle retrospective diagnostics, developers also require process-level control during agent execution.¹⁷ The AgentStepper interactive debugger adapts classical software debugging paradigms to LLM agents.¹⁷

AgentStepper instruments the agent's orchestration scaffold with explicit breakpoint hooks, allowing developers to inspect intermediate states.¹⁷ Key debugger mechanisms include:

- **Breakpoints:** Programmatic hooks (such as `begin_llm_query_breakpoint`) halt execution before critical LLM completions or tool calls, allowing developers to review parameters live.¹⁷
- **Stepwise Execution:** Developers can step through tool invocations sequentially, observing code edits in real time.¹⁷
- **Live Parameters Editing:** Developers can edit prompt inputs or tool arguments mid-run to test alternative approaches and prevent errors from propagating.¹⁷
- **Intermediate Diff Tracking:** The debugger records repository-level code changes at each step, displaying mutations as an interactive commit history.¹⁷

Integrating these interactive controls into agent workflows helps developers understand agent behavior and identify scaffold bugs, reducing developer frustration compared to inspecting raw logs.¹⁷

LLM-as-a-Judge Trajectory Evaluation

In automated evaluation pipelines, LLM-as-a-judge models assess the efficiency of tool-calling trajectories against reference standards.²⁰ Observability tools such as Arize Phoenix and LangSmith deploy prompt-based evaluation rubrics to score trajectories.²⁰

The evaluation prompt is configured with the user's initial input, the schemas of available tools, and the ordered sequence of tool calls extracted from execution spans.²¹ The judge evaluates the sequence based on three primary criteria²¹:

- **Logical Progression:** Does each step build logically toward resolving the user's query?²¹
- **Tool Selection Accuracy:** Did the agent invoke the correct tools for the task?²¹
- **Trajectory Efficiency:** Did the agent avoid unnecessary loops, redundant searches, or expensive detours?²¹

The judge outputs a binary correctness score and a detailed explanation, logging the diagnostic results back to the root span in the observability dashboard.²¹

Operational Diagnostics: Detecting Prompt Drift, Plan Drift, and Architectural Decay

AI Code Drift and Architectural Decay

As coding sessions extend across long horizons, agents suffer from progressive behavioral degradation known as session-level prompt drift.²² Over dozens of conversation turns, the accumulation of raw log files, tool parameters, and verbose explanations clutters the context window.²² This clutter causes the model to forget user-specified constraints, ignore previously flagged corrections, or fabricate agreements and APIs that do not exist.²²

At a codebase level, this failure mode scales up to become "AI code drift".²⁴ When a repository is maintained through a combination of manual developer patches, mismatched prompts from multiple team members, and varying generations of LLMs, the actual implementation rapidly diverges from the system's structural documentation.²⁴ Every prompt modification or partial refactoring introduces styling layers and logical contradictions that no single developer can explain, leading to context rot.²⁴

To eliminate drift entirely across the lifecycle of a codebase, teams are transitioning to GitOps-driven agent architectures, such as those defined by the Lyzr GitAgent specification.²⁵ Under this model, agent configurations, system prompts, and tool access schemas are treated strictly as infrastructure-as-code.²⁵ Prompt variations and behavioral updates cannot be executed silently in databases or third-party dashboards.²⁵ Instead, modifications must be submitted as declarative pull requests, subjected to human peer reviews, run through automated continuous integration validation pipelines, and deployed as immutable commits.²⁵ This architectural discipline ensures that the plain-language specification remains the single source of truth, aligning the codebase with design intentions.²⁴

Mathematical Drift Tracking

To detect session-level prompt drift programmatically, tools like Nautilus-Compass calculate real-time similarity scores.²² On every user prompt, the system computes the cosine similarity between the prompt embedding and a curated set of positive task patterns (desired task behaviors) and negative failure modes (known failures)²²:

$$\text{Drift Score} = \max_j (\text{sim}(P, A_j^+)) - \max_k (\text{sim}(P, A_k^-))$$

where P represents the active prompt vector, A^+ is the positive anchor set, A^- is the negative anchor set, and the similarity function is modeled as²²:

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

A negative drift score indicates that the user's instructions or the agent's responses are deviating toward known failure patterns.²² If this score falls below a critical threshold, the system triggers context alerts, injecting system directives to recalibrate the model's behavior.²²

Comparative Analysis of Observability Tooling

Observability architectures are divided into two distinct paradigms: Type 1 Enterprise Observability Platforms, which analyze production-scale API traces, and Type 2 Local Forensic Tools, which parse local session logs to optimize developer workflows.³

Type 1: Enterprise Observability Platforms

Type 1 platforms, exemplified by LangSmith, LangFuse, and Arize Phoenix, focus on production-scale, distributed trace analysis.²⁶ These systems ingest telemetry data over standardized protocols like OpenTelemetry and OpenInference, capturing structured execution spans across microservices and external API gateways.²⁶

The primary capabilities of Type 1 platforms include:

- **Production-Scale Monitoring:** Designed to handle heavy trace volumes, tracking latency spikes, error rates, and costs across production pipelines.²⁶
- **Programmatic Evaluation:** Run automated evaluation suites against production data, using LLM judges to evaluate prompt effectiveness and verify retrieval accuracy.²¹
- **Unsupervised Analytics:** Automatically cluster and analyze production traces to detect common execution patterns, failure modes, and anomalous behavior.²⁸

Type 2: Local Session Forensic Tools

Type 2 tools, including Agentsview, AiderDesk, ai-blame, and CCHV, are lightweight, local-first applications built to parse, search, and audit local session logs.³ These tools run fully offline on

developers' machines, parsing JSONL transcripts directly to render browsable, searchable interfaces.³

The primary advantages of Type 2 tools are:

- **Zero Workspace Pollution:** Run in isolated sandboxes or Git worktrees, allowing agents to test modifications and execute builds without touching active development branches.³⁴
- **Low-Latency Indexing:** Store session metadata in local SQLite or DuckDB databases, supporting fast full-text search across execution histories.³
- **Developer Controls:** Implement manual tool approval gates and conversational curation, allowing developers to approve shell commands, review file edits, and delete redundant messages to maintain clean context windows.³⁴

Evaluated Dimension	Type 1: Enterprise Observability Platforms (e.g., LangSmith, Phoenix)	Type 2: Local Session Forensic Tools (e.g., Agentsview, ai-blame)
Telemetry & Sync	Cloud-based or in-VPC deployments; ingest telemetry via OpenTelemetry/OTLP ²⁶	Local-first, fully offline; zero telemetry or external sync dependencies ³
Data Ingestion	Distributed API trace streaming with asynchronous ingestion engines ²⁶	File-system watchers that parse append-only local JSONL transcripts directly ³
Context Compilation	Two-tier skeleton compaction pipelines optimized for screening production runs ¹⁰	Algorithmic compactor hooks running locally with low latency ¹³
Developer Controls	Analytical dashboards, alerts, and programmatic evaluation runs ²⁸	Manual tool approval gates, isolated Git worktrees, and custom agent profiles ³⁴
Audit Granularity	Step-level and trace-level span metrics mapping execution costs ²⁶	Chronological action timelines and line-level model attribution embedded in files ³⁶

Strategic Engineering Blueprint for a Local Trace

Analysis Application

Integrating the technical documentation, empirical research, and competitor landscapes analyzed above provides a clear engineering blueprint for a local trace analysis application.

1. Ingestion Backend and File-System Watcher

To enable zero-dependency execution, the local viewer application must implement a fast file-system watcher (configured with a 500ms debounce interval) to monitor local agent session directories.³

Key backend specifications include:

- **Durable Storage:** Store parsed session metadata in a local SQLite database, using FTS5 virtual tables to support fast full-text search across message payloads.³
- **Database Modes:** Support analytical DuckDB push operations to enable team queries, and implement optional PostgreSQL sync daemons to push local sessions to a shared registry.³

2. Dual-Scope Detection System

The analytical engine must monitor agent behavior across both feedback-based and within-turn detection scopes:

Feedback-Based Scope

Parse subsequent user turns to identify signs of dissatisfaction.

The parser must flag user messages containing ⁹:

- **User Corrections:** Phrases such as "no", "incorrect", "fix this", or "compile failed".⁹
- **Failure Reports:** Stack traces, console log errors, or test suite outputs.⁹
- **Hard Interrupts:** Turn sequences that terminate abruptly before the agent completes its work, indicating the developer halted execution.⁹

Within-Turn Scope

Analyze private thinking logs and tool arrays within a single turn, independent of user feedback.

The engine must flag:

- **Tool Call Loops:** Identical tool names invoked with similar arguments across sequential spans.¹⁰
- **Argument Errors:** Parameters that violate tool schemas or generate terminal errors.¹⁰
- **Inference Spikes:** Turns where the model consumes high token volumes or spends excessive wall-clock time without generating file-system edits or terminal actions.¹⁰

3. Multi-Dimensional Cost Model

For each flagged failure, the engine must estimate its "price" along three dimensions: financial

costs, human attention costs, and agent runtime costs:

- **Financial Costs:** Calculate token expenditures per model call using LiteLLM pricing scales.³ The cost engine must account for prompt-caching economics, differentiating between cache creation tokens, cache read tokens, and standard input/output tokens to calculate the financial impact of each failure.³
- **Human Attention Cost:** Track the count of manual intervention cycles.⁸ This metric measures how many times the agent made the user provide corrections, resolve tool failures, or restart executions that could have been avoided with better path planning.⁸
- **Agent Runtime Cost:** Log wall-clock times of LLM completions and tool execution cycles.¹⁰ The cost engine calculates the time developers spent waiting for the agent to complete unproductive runs.¹⁰

4. Interactive UI and Live Alerting Engine

The local application should provide both a post-hoc analysis dashboard and a real-time hints engine to optimize developer workflows:

- **Interactive Session Browser:** A Svelte or React-based local web app displaying session timelines, interactive commit diffs, and token cost charts.³
- **Live CLI Alerts:** A terminal utility that monitors active coding sessions, running prompt-to-anchor cosine similarity checks on each turn.²² If similarity scores drop below defined thresholds, the system injects context alerts, prompting the developer to halt execution or edit the plan before the agent enters expensive error loops.¹⁷

Works cited

1. Building a session browser | Claude Cookbook, accessed June 15, 2026, <https://platform.claude.com/cookbook/claude-agent-sdk-05-building-a-session-browser>
2. Inside Claude Code: The Session File Format and How to Inspect It ..., accessed June 15, 2026, <https://databunny.medium.com/inside-claude-code-the-session-file-format-and-how-to-inspect-it-b9998e66d56b>
3. kenn-io/agentsview: Local-first session search, analytics ... - GitHub, accessed June 15, 2026, <https://github.com/kenn-io/agentsview>
4. Manage sessions - Claude Code Docs, accessed June 15, 2026, <https://code.claude.com/docs/en/sessions>
5. Agent Trace: Capturing the Context Graph of Code - Cognition, accessed June 15, 2026, <https://cognition.ai/blog/agent-trace>
6. cursor/agent-trace: A standard format for tracing AI ... - GitHub, accessed June 15, 2026, <https://github.com/cursor/agent-trace>
7. Trace any Copilot coding agent commit to its session logs - GitHub Changelog, accessed June 15, 2026, <https://github.blog/changelog/2026-03-20-trace-any-copilot-coding-agent-commit-to-its-session-logs/>

8. (PDF) SWE-chat: Coding Agent Interactions From Real Users in the Wild - ResearchGate, accessed June 15, 2026, https://www.researchgate.net/publication/404102665_SWE-chat_Coding_Agent_Interactions_From_Real_Users_in_the_Wild
9. SWE-chat: Coding Agent Interactions From Real Users in the Wild - arXiv, accessed June 15, 2026, <https://arxiv.org/html/2604.20779v1>
10. How We Built LangSmith Engine, Our Agent for Improving Agents - LangChain, accessed June 15, 2026, <https://www.langchain.com/blog/how-we-built-langsmith-engine-our-agent-for-improving-agents>
11. [2603.29678] View-oriented Conversation Compiler for Agent Trace Analysis - arXiv, accessed June 15, 2026, <https://arxiv.org/abs/2603.29678>
12. View-oriented Conversation Compiler for Agent Trace Analysis - arXiv, accessed June 15, 2026, <https://arxiv.org/html/2603.29678v1>
13. sting8k/pi-vcc: Smart, Fast & Lossless session compaction for Pi. No LLM calls - produces structured, transcript-preserving summaries using pure extraction and formatting. - GitHub, accessed June 15, 2026, <https://github.com/sting8k/pi-vcc>
14. monotykamary/pi-vcc · Packages - Pi Coding Agent, accessed June 15, 2026, <https://pi.dev/packages/@monotykamary/pi-vcc>
15. CodeTracer: Towards Traceable Agent States - arXiv, accessed June 15, 2026, <https://arxiv.org/html/2604.11641v3>
16. Paper page - CodeTracer: Towards Traceable Agent States - Hugging Face, accessed June 15, 2026, <https://huggingface.co/papers/2604.11641>
17. AgentStepper: Interactive Debugging of Software Development Agents - arXiv, accessed June 15, 2026, <https://arxiv.org/html/2602.06593v1>
18. sola-st/AgentStepper - GitHub, accessed June 15, 2026, <https://github.com/sola-st/AgentStepper>
19. AgentStepper: Interactive Debugging of Software Development Agents - arXiv, accessed June 15, 2026, <https://arxiv.org/pdf/2602.06593>
20. How to evaluate your agent with trajectory evaluations - Docs by LangChain, accessed June 15, 2026, <https://docs.langchain.com/langsmith/trajectory-evals>
21. Agent Trajectory Evaluations - Arize AX Docs, accessed June 15, 2026, <https://arize.com/docs/ax/evaluate/evaluators/trace-and-session-evals/trace-level-evaluations/agent-trajectory-evaluations>
22. Nautilus Compass: Black-box Persona Drift Detection for Production LLM Agents - arXiv, accessed June 15, 2026, <https://arxiv.org/html/2605.09863v1>
23. What I Learned from Running Command-A Reasoning 08-2025 Inside an Aider Coding Loop | IoFT LLC, accessed June 15, 2026, <https://loftllc.dev/en/docs/tech/llm-research/using-command-a-reasoning-in-an-aiders-coding-loop/>
24. The Real Cost of AI-Generated Code Drift, and How to Stop It | MindStudio, accessed June 15, 2026, <https://www.mindstudio.ai/blog/ai-generated-code-drift-cost-analysis>
25. prompt drift is an operations nightmare. we started using gitops for our agents. - Reddit, accessed June 15, 2026,

https://www.reddit.com/r/devops/comments/1u1el4c/prompt_drift_is_an_operational_nightmare_we/

26. Best AI Agent Evaluation Tools for Production Teams (2026) | Augment Code, accessed June 15, 2026, <https://www.augmentcode.com/tools/best-ai-agent-evaluation-tools>
27. Agent Graphs - Langfuse, accessed June 15, 2026, <https://langfuse.com/docs/observability/features/agent-graphs>
28. LangSmith: AI Agent & LLM Observability Platform - LangChain, accessed June 15, 2026, <https://www.langchain.com/langsmith/observability>
29. Best LLM tracing tools for multi-agent systems (2026 review) - Articles - Braintrust, accessed June 15, 2026, <https://www.braintrust.dev/articles/best-llm-tracing-tools-2026>
30. Add Observability to Your Open Agent Spec Agents with Arize Phoenix, accessed June 15, 2026, <https://arize.com/blog/add-observability-to-your-open-agent-spec-agents-with-arize-phoenix/>
31. Tracing & Evaluating a Custom Support Agent - Arize AX Docs, accessed June 15, 2026, <https://arize.com/docs/ax/cookbooks/agents/tracing-and-evaluating-agents>
32. The Agent Improvement Loop Starts with a Trace - LangChain, accessed June 15, 2026, <https://www.langchain.com/blog/traces-start-agent-improvement-loop>
33. Human judgment in the agent improvement loop - LangChain, accessed June 15, 2026, <https://www.langchain.com/blog/human-judgment-in-the-agent-improvement-loop>
34. hotovo/aider-desk: Platform for AI-powered software ... - GitHub, accessed June 15, 2026, <https://github.com/hotovo/aider-desk>
35. Claude Code History Viewer: Every Tool Compared & Ranked ..., accessed June 15, 2026, <https://easyclaw.com/blog/knowledge/claude-code-history-viewer-compared/>
36. ai-blame - AI4Curation, accessed June 15, 2026, <https://ai4curation.io/ai-blame/>
37. Task Management - AiderDesk, accessed June 15, 2026, <https://aiderdesk.hotovo.com/docs/features/tasks>
38. Evaluating Deep Agents using LangSmith on AWS | Artificial Intelligence, accessed June 15, 2026, <https://aws.amazon.com/blogs/machine-learning/evaluating-deep-agents-using-langsmith-on-aws/>
39. OpenHands/trajectory-visualizer - GitHub, accessed June 15, 2026, <https://github.com/OpenHands/trajectory-visualizer>
40. Oversee a prior art search AI agent with human-in-the-loop by using LangGraph and watsonx.ai - IBM, accessed June 15, 2026, <https://www.ibm.com/think/tutorials/human-in-the-loop-ai-agent-langgraph-watsonx-ai>